becomes `[0,1]` for an action space of [down, up]. The grid only contains a series of binary digits representing the spin of each agent, but we need to turn those binary digits into action vectors and then take the outer product to get a joint-action vector for the Q function. In listing 9.7 we put some of the pieces we've made together to create a new grid and a set of parameter vectors that, in effect, comprises the set of agents on the grid.

Listing 9.7. The 1D Ising model: Initialize the grid

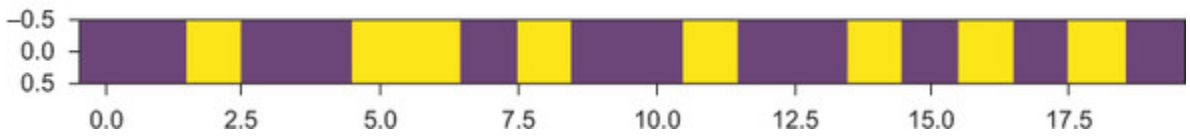
```
plt.figure(figsize=(8,5))
size = (20,)                                                    1
hid_layer = 20                                                  2
params = gen_params(size[0],4*hid_layer+hid_layer*2)            3
grid = init_grid(size=size)
grid_ = grid.clone()                                            4
print(grid)
plt.imshow(np.expand_dims(grid,0))
```

- ***1*** Sets the total size of the grid to be a 20-length vector
- ***2*** Sets the size of the hidden layer. Our Q function is just a 2-layer neural network, so there's only one hidden layer.
- ***3*** Generates a list of parameter vectors that will parameterize the Q function
- ***4*** Makes a clone of the grid (for reasons that will become clear in the main training loop)

If you run the listing 9.7 code, you should get something like figure 9.14, but yours will look different since it is randomly initialized.

```
tensor([0, 0, 1, 0, 0, 1, 1, 0, 1, 0, 0, 1, 0, 0, 1, 0, 1, 0, 1,
0],
       dtype=torch.uint8)
```

Figure 9.14. A 1D Ising model of electrons arranged in a single row.



You will notice that the spins are pretty randomly distributed between up (1) and down (0). When we train our Q function, we expect to get the spins